

Kokoro Expressive Audiobook TTS — Design Specification

Date: 2026-10-06

Status: Draft — Pending User Approval

Scope: Fine-tuned Kokoro-82M with ElevenLabs-style expression tags + Gradio Audiobook UI

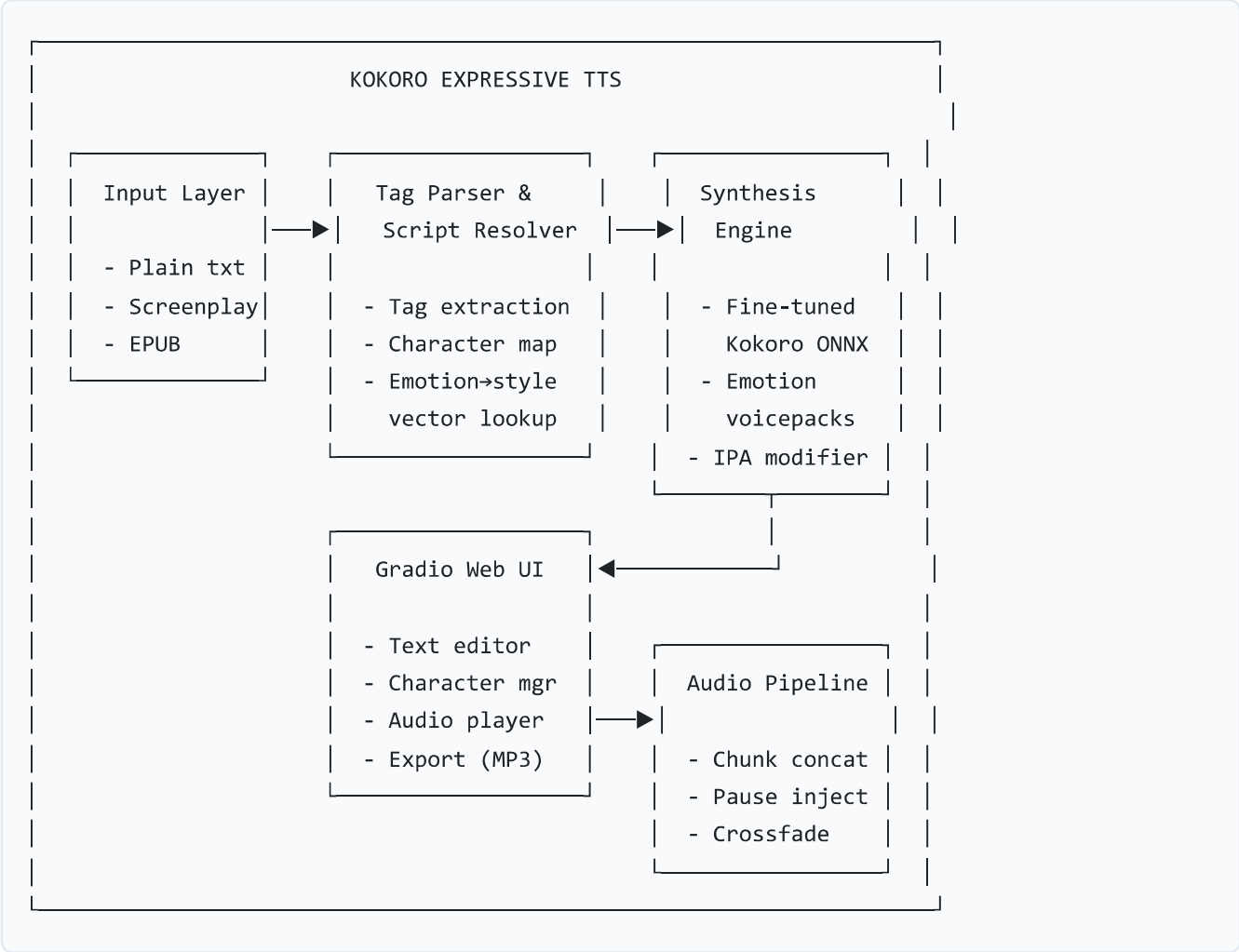
1. Problem Statement & Background

Kokoro-82M is a fast, high-quality, open-weight TTS model based on StyleTTS 2. In its default form, it produces neutral-to-pleasant speech but has **no built-in understanding of emotional context or expression directives**. The goal is to transform it into an ElevenLabs-grade expressive audiobook TTS engine where authors can write:

```
[narrator] It was a dark and stormy night.  
[sad] She whispered, "I don't know how much longer I can do this."  
[gasps] A shadow appeared at the door.  
[angry] "Get out!" he shouted.
```

...and receive audio where each line is delivered with the correct emotion, vocal quality, pacing, and character voice — fully rendered on a CPU.

2. Architecture Overview



3. System Components

3.1 Component Map

Component	Purpose	Technology
Tag Parser	Extracts emotion/action tags from text, maps to synthesis params	Pure Python, regex
Script Resolver	Handles multiple input formats (plain text, screenplay, EPUB)	ebooklib , regex
Emotion Voicepack Store	Per-emotion style vectors (.npz or .pt files)	NumPy / PyTorch
IPA Emotion Modifier	Adds pitch/stress markers to phonemes based on emotion	String manipulation
Synthesis Engine	ONNX inference with fine-tuned Kokoro weights	kokoro-onnx
Audio Pipeline	Chunk concatenation, pause injection, crossfading	soundfile , NumPy
Training Pipeline	Fine-tune Kokoro on emotional speech datasets	kikiri-tts , Colab
Gradio UI	Browser-based interface for the audiobook tool	gradio

4. Training Pipeline (Colab/Kaggle)

This is the most critical component. Fine-tuning produces emotion-aware style vectors that the inference engine uses.

4.1 Training Strategy

We use the `semidark/kikiri-tts` pipeline (the only proven community harness for Kokoro-82M training) with emotional speech datasets. The goal is NOT to retrain the entire model from scratch — that costs ~\$1,000 in GPU time. Instead we:

1. **Start from the pretrained Kokoro-82M checkpoint** (`hexgrad/Kokoro-82M`)
2. **Fine-tune only the ProsodyPredictor and style-conditioning layers** while freezing the iSTFTNet vocoder and PL-BERT encoder
3. **Train on emotional speech datasets** labeled by emotion category

4. **Extract per-emotion style vectors** from the trained model using reference audio from each emotion class

This is far more efficient than full retraining:

- Full training: ~1,000 A100 GPU-hours
- Targeted fine-tuning (frozen backbone): ~20-50 A100-equivalent GPU-hours → **~8-20 hours on free Colab T4**

4.2 Training Datasets

Dataset	Size	Emotions	License
ESD (Emotional Speech Database)	35,000 utterances, 10 speakers	Neutral, Happy, Angry, Sad, Surprised	Free for research
RAVDESS	7,356 utterances, 24 actors	Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised	Creative Commons
Expresso (Meta)	26 hours, 4 speakers	26 styles: whisper, sigh, laugh, confused, happy, sad, angry, etc.	CC-BY 4.0
LJSpeech (neutral baseline)	24 hrs, 1 speaker	Neutral	Public Domain

Total training corpus target: ~60-70 hours — manageable for fine-tuning in multiple Colab sessions.

4.3 Data Preparation Pipeline

```
Raw Audio (WAV) → Resample to 24kHz → Loudness Normalize (-23 LUFS)
↓
Silence Trim (top_db=30) → Segment to <10s clips → Quality Filter (SNR > 20dB)
↓
Transcription → IPA Phonemization (misaki/espeak-ng) → Validate against vocab
↓
Generate metadata: audio_path | ipa_phonemes | speaker_id | emotion_id
↓
Split: 95% train / 5% validation (stratified by emotion + speaker)
```

Metadata format (`train_list.txt`):

```
data/esd/0001_happy_001.wav|hɛlou wɜːld|speaker_0011|emotion_happy
data/ravdess/03-01-05-01-01-01.wav|ðɪs ɪz ɛŋɡrɪ|speaker_0023|emotion_angry
```

4.4 Emotion Token Injection (Key Innovation)

Standard Kokoro uses only phoneme tokens (IDs 1–177). We extend the vocabulary with **emotion conditioning tokens** prepended to each utterance:

```
python
EMOTION_TOKENS = {
    "neutral": 178,
    "happy": 179,
    "sad": 180,
    "angry": 181,
    "fearful": 182,
    "excited": 183,
    "calm": 184,
    "nervous": 185,
    "whisper": 186,
    "shout": 187,
    "laugh": 188,
    "cry": 189,
    "sigh": 190,
    "gasp": 191,
    "dramatic": 192,
    "sarcastic": 193,
    "monotone": 194,
    "hesitant": 195,
    "confident": 196,
}
```

At training time: `input_ids = [[0, EMOTION_TOKEN, *phoneme_tokens, 0]]`

At inference time, the tag parser extracts the emotion tag and prepends the corresponding token, so the model itself conditions on the emotion *before* generating any audio.

4.5 Two-Stage Training Schedule

Stage 1: Acoustic Pre-training (Colab Session 1 — ~4-6 hours)

- **Frozen layers:** iSTFTNet vocoder, PL-BERT (CustomAlbert)
- **Trainable layers:** TextEncoder , ProsodyPredictor , emotion token embeddings
- **Loss functions:**
 - L1 Mel reconstruction loss
 - Multi-Resolution STFT loss
 - Duration (alignment) loss

- F0/energy prediction loss
- **Batch size:** 4 (T4-friendly)
- **LR:** 1e-4 with cosine decay

Stage 2: Adversarial Prosody Training (Colab Session 2-3 — ~8-12 hours)

- **Frozen layers:** iSTFTNet vocoder only
- **Trainable:** All encoder/predictor layers + emotion embeddings
- **Additional losses added:**
 - Multi-Period Discriminator (MPD) adversarial loss
 - Multi-Resolution STFT Discriminator (MRD) loss
 - WavLM-based perceptual discriminator loss (SLM loss)
 - Feature matching loss
- **LR:** 1e-5 with warmup

4.6 Voicepack Extraction

After training, emotion voicepacks are extracted by running the model's style encoder over reference audio clips from each emotion category:

```
python
# For each emotion class:
# 1. Select 20-50 representative utterances from validation set
# 2. Run through StyleEncoder to get style vectors
# 3. Average across utterances → one (511, 1, 256) tensor per emotion
# 4. Save as emotions/happy.npy, emotions/sad.npy, etc.
```

This gives us **per-emotion style tensors** that the inference engine can dynamically select based on the detected tag.

5. Tag Parser Engine

5.1 Supported Tag Formats

Format A: Inline Square Brackets (ElevenLabs-style)

```
[happy] She smiled at the camera.  
[whispering, nervous] "Did you hear that?" he asked.  
[long pause] The room went silent.
```

Format B: Screenplay Format

```
ALICE (angry): I can't believe you did that!  
BOB (sad, quietly): I'm sorry. I didn't mean to.  
NARRATOR (dramatic): The clock struck midnight.
```

Format C: EPUB/TXT Import

- Upload EPUB → extract chapters → display in editor
- User annotates emotion tags in the Gradio UI
- Tags stored as metadata alongside text

5.2 Tag-to-Parameter Mapping

Each tag maps to a **synthesis parameter bundle**:

```

python
EMOTION_PROFILES = {
    "happy": {
        "emotion_token": EMOTION_TOKENS["happy"],
        "speed": 1.1,
        "style_blend": {"af_heart": 0.6, "af_bella": 0.4}, # energetic voices
        "ipa_prefix": "^", # rising pitch marker
        "pause_before": 0.0,
        "pause_after": 0.05,
    },
    "sad": {
        "emotion_token": EMOTION_TOKENS["sad"],
        "speed": 0.85,
        "style_blend": None, # use pure emotion voicepack
        "ipa_prefix": "\", # falling pitch
        "pause_before": 0.1,
        "pause_after": 0.15,
    },
    "whisper": {
        "emotion_token": EMOTION_TOKENS["whisper"],
        "speed": 0.9,
        "style_blend": None,
        "ipa_prefix": None,
        "pause_before": 0.05,
        "pause_after": 0.05,
        "post_process": "reduce_volume_db(-8)", # DSP whisper simulation
    },
    "shout": {
        "emotion_token": EMOTION_TOKENS["shout"],
        "speed": 1.2,
        "ipa_prefix": "↑",
        "pause_before": 0.0,
        "pause_after": 0.1,
        "post_process": "increase_volume_db(+4)",
    },
    "laugh": {
        "emotion_token": EMOTION_TOKENS["laugh"],
        "speed": 1.15,
        "inject_sfx": "sfx/laugh.wav", # inject non-verbal audio clip
        "pause_before": 0.05,
        "pause_after": 0.1,
    },
    "gasp": {
        "emotion_token": EMOTION_TOKENS["gasp"],
        "inject_sfx": "sfx/gasp.wav",
    }
}

```

```

        "pause_before": 0.0,
        "pause_after": 0.05,
    },
    "long pause": {
        "inject_silence_ms": 1200,
    },
    "pause": {
        "inject_silence_ms": 500,
    },
    # ... (full table for all 30+ tags)
}

```

5.3 Tag Parser Logic

```

python
import re

TAG_PATTERN = re.compile(r'\[[^\]]+\]\s*')
SCREENPLAY_PATTERN = re.compile(
    r'^(?P<character>[A-Z][A-Z\s]+\s*\((?P<emotion>[^\)]+\)\s*:\s*(?P<text>.+)$'
)

class TagParser:
    def parse(self, raw_text: str) -> list[SpeechSegment]:
        """
        Returns a list of SpeechSegment objects, each with:
        - text: str (text to synthesize)
        - emotion: str
        - character: str | None
        - params: EmotionProfile
        """

```

5.4 Non-Verbal Sound Effects (SFX Library)

For action tags that are hard to synthesize (laugh, gasp, sigh, cry), we maintain a small library of recorded or synthesized SFX clips that are **injected into the audio timeline**:

```
sfx/  
  laugh_female.wav  
  laugh_male.wav  
  gasp.wav  
  sigh.wav  
  cry_sob.wav  
  deep_breath.wav  
  throat_clear.wav
```

These are concatenated into the audio at the correct position by the Audio Pipeline.

6. Multi-Character Voice System

6.1 Character Voice Registry

Each character in a book can be assigned a distinct Kokoro voice (from the 54 available voices) plus an optional emotion modifier:

```
python  
CHARACTER_REGISTRY = {  
    "NARRATOR": {  
        "base_voice": "af_heart",    # calm, authoritative  
        "gender": "female",  
        "sfx_set": "female",  
    },  
    "ALICE": {  
        "base_voice": "af_bella",    # younger, energetic  
        "gender": "female",  
        "sfx_set": "female",  
    },  
    "BOB": {  
        "base_voice": "am_michael",  # male narrator  
        "gender": "male",  
        "sfx_set": "male",  
    },  
}
```

6.2 Emotion × Character Combination

At synthesis time, the final style vector is computed as:

```
final_style = emotion_voicepack[emotion] × α + base_voice[character] × (1-α)
```

Where α (emotion strength, 0.0–1.0) is configurable per-tag. This blends the character's timbre with the emotion's prosodic pattern.

7. Gradio Web UI

7.1 UI Sections

 Kokoro Expressive Audiobook TTS

[Tab: Editor] [Tab: Characters] [Tab: Export] [Tab: Train]

EDITOR TAB:

Text Input

[Import EPUB] [.txt]

NARRATOR: It was a dark and stormy night.
[sad] "I can't go on."

 Tag Helper

Audio Output

▶ Play

↓ Download MP3

1:23/4:50

[Segment List]

✓ Segment 1 - 2.3s

✓ Segment 2 - 1.8s

[Generate ▶] Speed: [1.0]

CHARACTERS TAB:

[+ Add Character]

NARRATOR	Voice: af_heart	[Preview]
ALICE	Voice: af_bella	[Preview]
BOB	Voice: am_michael	[Preview]

7.2 Tag Helper Widget

An inline tag-picker dropdown that inserts tags at cursor position — so the user doesn't need to memorize tag names.

7.3 Segment-by-Segment Generation

For long texts, the UI generates audio segment by segment and streams them to the player as they complete, rather than waiting for the full book to process.

8. ONNX Export (CPU Inference)

After fine-tuning, the PyTorch model must be exported back to ONNX for use with `kokoro-onnx`:

```
python
import torch

model.eval()
dummy_tokens = torch.zeros(1, 10, dtype=torch.long)
dummy_style = torch.zeros(1, 256, dtype=torch.float32)
dummy_speed = torch.tensor([1.0])

torch.onnx.export(
    model,
    (dummy_tokens, dummy_style, dummy_speed),
    "kokoro-expressive-v1.0.onnx",
    input_names=["tokens", "style", "speed"],
    output_names=["audio"],
    dynamic_axes={
        "tokens": {1: "seq_len"},
        "audio": {1: "num_samples"},
    },
    opset_version=17,
)
```

Post-export, apply **INT8 quantization** for CPU efficiency:

```
python
from onnxruntime.quantization import quantize_dynamic
quantize_dynamic(
    "kokoro-expressive-v1.0.onnx",
    "kokoro-expressive-v1.0.int8.onnx",
    weight_type=QuantType.QInt8
)
```

This reduces model size from ~320MB → ~80MB and gives ~2-4× CPU speedup.

9. Project Structure

```
kokoro-tts/
├── venv/                                # Existing virtual environment
│
├── app/                                # Main application code
│   ├── __init__.py
│   ├── app.py                          # Gradio UI entrypoint
│   ├── parser/
│   │   ├── tag_parser.py              # Tag extraction & format detection
│   │   ├── script_resolver.py         # EPUB/TXT/screenplay import
│   │   └── emotion_profiles.py        # Tag → parameter mapping table
│   ├── synthesis/
│   │   ├── engine.py                  # Kokoro ONNX wrapper
│   │   ├── voicepack_store.py         # Emotion + character voice management
│   │   ├── ipa_modifier.py            # Phoneme-level pitch/stress injection
│   │   └── audio_pipeline.py          # Chunking, concat, crossfade, SFX
│   └── characters/
│       └── registry.py                # Character→voice mapping
│
├── emotions/                           # Extracted emotion voicepacks
│   ├── happy.npy
│   ├── sad.npy
│   ├── angry.npy
│   └── ... (one per emotion)
│
├── sfx/                                # Non-verbal sound effect clips
│   ├── laugh_female.wav
│   ├── gasp.wav
│   └── ...
│
├── models/                             # Model weight files
│   ├── kokoro-v1.0.onnx               # Base Kokoro (or fine-tuned)
│   ├── kokoro-expressive.onnx        # Fine-tuned (after training)
│   └── voices-v1.0.bin                # Base voice packs
│
├── training/                           # Training pipeline (run on Colab)
│   ├── README_COLAB.md               # Step-by-step Colab guide
│   ├── 01_prepare_data.py            # Dataset download & preprocessing
│   ├── 02_stage1_train.py            # Stage 1: acoustic pretraining
│   ├── 03_stage2_train.py            # Stage 2: adversarial training
│   ├── 04_extract_voicepacks.py      # Extract emotion style vectors
│   ├── 05_export_onnx.py             # PyTorch → ONNX export
│   └── configs/
│       └── stage1_config.yaml
```

```
|      └─ stage2_config.yaml
|
| └─ docs/
|   └─ superpowers/
|     └─ specs/
|       └─ 2026-10-06-kokoro-expressive-audiobook-tts-design.md
|
| └─ requirements.txt
| └─ README.md
```

10. Implementation Phases

Phase 1: Inference App (CPU, No Training) — Week 1

Goal: Working Gradio app with tag parsing using the base Kokoro model

- Download base Kokoro model weights (`kokoro-v1.0.onnx` , `voices-v1.0.bin`)
- Build `TagParser` — inline tags, screenplay format, EPUB import
- Build `EmotionProfiles` — all 30+ tags mapped to speed/style/IPA params
- Build `SynthesisEngine` — wraps `kokoro-onnx` with per-segment control
- Build `AudioPipeline` — crossfading, pause injection, SFX injection
- Build `CharacterRegistry` — multi-voice management
- Build Gradio UI — Editor, Characters tabs, audio player
- Add SFX library (laugh, gasp, sigh — short recorded clips)

Phase 2: Training Data Preparation (Colab) — Week 2

Goal: Clean, formatted dataset ready for fine-tuning

- Download ESD, RAVDESS, Espresso datasets
- Audio preprocessing: resample → normalize → trim → segment
- IPA phonemization aligned with Kokoro's `misaki` vocab
- Quality filtering (SNR, duration bounds)
- Generate `train_list.txt` and `val_list.txt`
- Set up `kikiri-tts` repo on Colab/Kaggle

Phase 3: Fine-Tuning (Colab/Kaggle) — Weeks 3-4

Goal: Emotion-aware Kokoro model weights

- Set up kikiri-tts training environment on Colab
- Patch emotion token injection into vocabulary
- Run Stage 1 training (~4-6 hrs on T4)
- Run Stage 2 adversarial training (~8-12 hrs across sessions)
- Evaluate: MOS score, emotion classification accuracy
- Extract per-emotion voicepacks (`.npy` files)
- ONNX export + INT8 quantization

Phase 4: Integration & Polish — Week 5

Goal: Fine-tuned model integrated into app with full feature set

- Swap base ONNX model for fine-tuned model in app
- Update emotion profiles to use fine-tuned voicepacks
- Full EPUB audiobook pipeline (chapter detection, progress bar)
- MP3 export with chapter markers
- Test with full audiobook text (> 10,000 words)
- Performance benchmarking (CPU inference speed)

11. Key Technical Risks & Mitigations

Risk	Likelihood	Mitigation
Colab T4 session limits (12hr/session)	High	Split training into stages; save checkpoints every 100 steps; use Kaggle as backup
Emotion fine-tuning degrades base voice quality	Medium	Keep base Kokoro ONNX as fallback; use low LR (1e-5) in Stage 2
kikiri-tts incompatibility with Kokoro v1.0 weights	Medium	Pin to known-working commit; test on small dataset first
ONNX export fails with custom emotion tokens	Medium	Test export after adding tokens before full training run
Espresso/ESD data alignment errors	Low	Validate every sample with forced alignment before training

12. Verification Plan

Per-Phase Verification

- **Phase 1:** Generate test audio for all 30+ tags, confirm prosody differences are audible
- **Phase 2:** Validate dataset: run phonemization on 100 samples, check vocab coverage
- **Phase 3:** Loss curves converging (mel loss < 0.4), MOS ≥ 3.8, human AB test vs. base model
- **Phase 4:** Full audiobook test (Pride and Prejudice Chapter 1, ~2,500 words), CPU real-time factor < 3×

Emotion Quality Benchmark

Run each of 19 emotion tags on 10 standardized test sentences; verify via:

1. Automatic emotion classifier (SER model) correctly identifies synthesized emotion (target: >70% accuracy)
2. A/B listening test (informal) comparing to base Kokoro neutral output

This document will be updated as implementation progresses.